20250818日报

今日工作内容

1. 修改在线服务日志扫描的逻辑，之前是直接扫描的trpc-go-yaml和trpc-python-yaml，但是实际上启动配置文件名称可能有很多种的；

   修改方案：通过api接口读取正式服的所有配置（不一定是启动配置），然后再对每个配置里的内容进行解析，如果其结构能解析为如下结构体，那么就一定是启动配置文件：

```
        Global struct {
            Namespace string `yaml:"namespace"`
        } `yaml:"global"`
        Server struct {
            Service []struct{} `yaml:"service"`
        } `yaml:"server"`
    }
```

   相应新增的代码逻辑如下：

```
// GetReleaseConfigName 获取指定gdp服务的正式服服务的所有配置名称 ConfigName
func GetReleaseConfigNames(apiAddr string, projectName string, service string) []string {
	var configNames []string
	page := 1
	for {
		apiURL := fmt.Sprintf("%s/v1/project/%s/paas/%s/configs/startup/configs?searchKey=&pageNow=%d&pageSize=10",
			apiAddr, projectName, service, page)
		headers := getAuthHeaders(apiAddr)
		req, _ := http.NewRequest("GET", apiURL, nil)
		for k, v := range headers {
			req.Header.Add(k, v)
		}
		resp, err := http.DefaultClient.Do(req)
		if err != nil {
			log.Errorf("request failed: %v", err)
			break
		}
		defer resp.Body.Close()
		body, _ := io.ReadAll(resp.Body)
		var result map[string]interface{}
		if err := json.Unmarshal(body, &result); err != nil {
			break
		}
		if code, ok := result["code"].(float64); !ok || int(code) != 0 {
			break
		}
		data, _ := result["data"].(map[string]interface{})
		dataList, _ := data["dataList"].([]interface{})
		if len(dataList) == 0 || dataList == nil {
```

2. 为思航哥他们提供hok中所有正式服服务的启动配置文件；

```
func main() {
    configDir := "hokconfigs/"
    services := task.ListAllServices(apiHOKAddr, projectHOKName)
    for _, service := range services {
        serviceName := service["paas_name"].(string)
        // 获取该服务所有正式配置名称
        serviceConfigName := task.GetReleaseConfigNames(apiHOKAddr, projectHOKName, serviceName)
        for _, configName := range serviceConfigName {
            // 获取该配置名称的所有生效版本
            configVersions := task.GetStartupConfigList(apiHOKAddr, projectHOKName, serviceName, configName)
            for _, config := range configVersions {
                // 获取该版本的具体配置
                version := config["version"].(string)
                data := task.SearchOneStartupConfig(apiHOKAddr, projectHOKName, serviceName, version)
                var parsedConfig ConfigNameConfig
                if err := yaml.Unmarshal([]byte(data), &parsedConfig); err != nil {
                    log.Errorf("yaml unmarshal failed: %v", err)
                } else {
                    log.Infof("Namespace: %v", parsedConfig.Global.Namespace)
                    if parsedConfig.Global.Namespace != "" && parsedConfig.Server.Service != nil {
                        safeFileName := serviceName + "_" + configName + "_" + version + ".yaml"
                        filePath := filepath.Join(configDir, safeFileName)
                        if err := os.WriteFile(filePath, []byte(data), 0644); err != nil {
                            log.Errorf("write file failed %s: %v", filePath, err)
                        } else {
                            log.Infof("write file success: %s", filePath)
                        }
```

生成的yaml效果为：服务名_启动配置名称_版本名.yaml

明日待办：

1. 将修改后的代码部署到王者、hok、aov上，并在grafana上完成告警监控；